Question 1

For each spatial dimension of a transposed convolution (dilation = 1):
$O = (I - 1)S - 2P + K + OP$.

With $I = 8$, $K = 4$, $S = 2$, $P = 1$ and $OP = 1$:
$O = (8 - 1)(2) - 2(1) + 4 + 1 = 14 - 2 + 4 + 1 = 17$.
The output spatial size is **17 x 17**.

Question 2

Keeping the Question 1 values fixed and increasing stride from 2 to 3:
$O = (8 - 1)(3) - 2(1) + 4 + 1 = 21 - 2 + 4 + 1 = 24$.
The output becomes **24 x 24**, an increase of 7 in each dimension.
In general, increasing stride by 1 increases each output dimension by $I - 1$.

Question 3

For a 2D transposed convolution with dilation 1, apply this formula separately to height and width:
$O = (I - 1)S - 2P + K + OP$.
Here I is input size, K kernel size, S stride, P padding on each side, and OP output padding. If these parameters differ by axis, use the corresponding height and width values in each calculation.

Question 4

Choose **kernel size 2 x 2, stride 2, padding 0, output padding 0**.
$O = (16 - 1)(2) - 2(0) + 2 + 0 = 30 + 2 = 32$.
Thus, a 16 x 16 input is upsampled to **32 x 32**.

Question 5

For the mini-batch [6, 8, 10, 6], use the batch (population) variance:
Mean: mu = (6 + 8 + 10 + 6) / 4 = 30 / 4 = 7.5.
Deviations: [-1.5, 0.5, 2.5, -1.5].
Variance: var = (2.25 + 0.25 + 6.25 + 2.25) / 4 = 11 / 4 = 2.75.
Standard deviation: sqrt(2.75) = sqrt(11) / 2, approximately 1.6583.

With no learned scale or shift, x_hat = (x - mu) / sqrt(var).
The normalized output is [-3/sqrt(11), 1/sqrt(11), 5/sqrt(11), -3/sqrt(11)],
approximately **[-0.9045, 0.3015, 1.5076, -0.9045]**.
These exact normalized values have mean 0 and population variance 1.
For this hand calculation, epsilon is omitted. Practical BatchNorm layers add a small epsilon inside the square root for numerical stability.

Question 6

ReLU(x) = max(0, x): x for x >= 0; 0 for x < 0.
LeakyReLU(x) = x for x >= 0; alpha*x for x < 0, where alpha is a small positive constant (0.2 in this assignment).
The key difference is the negative-side slope: 0 for ReLU versus alpha for LeakyReLU.

Question 7

LeakyReLU keeps a nonzero gradient for negative inputs, reducing the "dying ReLU" problem. This lets negative-input neurons continue learning and can improve gradient flow in deep networks, including GAN discriminators.